You Can't Eat Your Cake and Have It Too: The Performance Degradation of LLMs with Jailbreak Defense

Wuyuao Mai*
Fudan University
Shanghai, China
maiwuyuao20@fudan.edu.cn

Geng Hong*
Fudan University
Shanghai, China
ghong@fudan.edu.cn

Pei Chen
Fudan University
Shanghai, China
peichen19@fudan.edu.cn

Xudong Pan
Fudan University
Shanghai, China
xdpan@fudan.edu.cn

Baojun Liu
Tsinghua University
Beijing, China
lbj@tsinghua.edu.cn

Yuan Zhang
Fudan University
Shanghai, China
yuanxzhang@fudan.edu.cn

Haixin Duan
Quancheng Laboratory
Tsinghua University
Beijing, China
duanhx@tsinghua.edu.cn

Min Yang
Fudan University
Shanghai, China
m_yang@fudan.edu.cn

Abstract

With the rise of generative large language models (LLMs) like LLaMA and ChatGPT, these models have significantly transformed daily life and work by providing advanced insights. However, as jailbreak attacks continue to circumvent built-in safety mechanisms, exploiting carefully crafted scenarios or tokens, the safety risks of LLMs have come into focus. While numerous defense strategies—such as prompt detection, modification, and model fine-tuning—have been proposed to counter these attacks, a critical question arises: do these defenses compromise the utility and usability of LLMs for legitimate users? Existing research predominantly focuses on the effectiveness of defense strategies without thoroughly examining their impact on performance, leaving a gap in understanding the trade-offs between LLM safety and performance.

Our research addresses this gap by conducting a comprehensive study on the utility degradation, safety elevation, and exaggerated-safety escalation of LLMs with jailbreak defense strategies. We propose ***USEBench***, a novel benchmark designed to evaluate these aspects, along with ***USEIndex***, a comprehensive metric for assessing overall model performance. Through experiments on seven state-of-the-art LLMs, we found that mainstream jailbreak defenses fail to ensure both safety and performance simultaneously. Although model-finetuning performs the best overall, their effectiveness varies across LLMs. Furthermore, vertical comparisons reveal that developers commonly prioritize performance over safety when iterating or fine-tuning their LLMs.

CCS Concepts

• **Security and privacy → Web application security**.

Keywords

LLM Jailbreak, Jailbreak Evaluation, LLM Performance Downgrade, LLM Benchmark

ACM Reference Format:
Wuyuao Mai, Geng Hong, Pei Chen, Xudong Pan, Baojun Liu, Yuan Zhang, Haixin Duan, and Min Yang. 2025. You Can't Eat Your Cake and Have It Too: The Performance Degradation of LLMs with Jailbreak Defense. In *Proceedings of the ACM Web Conference 2025 (WWW '25), April 28–May 2, 2025, Sydney, NSW, Australia.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3696410.3714632

1 Introduction

With the emergence of generative large language models (LLMs), such as LLaMA[45] and ChatGPT[12], there has been a transformative impact on both daily life and work. We are amazed by how a web AI assistant can offer insights into the complex mysteries of human society. Concurrently, the safety issues surrounding LLMs are receiving increasing attention. The inherent safety guardrails set by developers for LLMs are often circumvented through jailbreak attacks. By creating seemingly safe task scenarios [31] or selecting carefully crafted tokens [21, 30], attackers exploit LLMs to generate illegal content that infringes on copyright, promotes racial discrimination, and may cause harm to individuals and society.

In response to the challenges posed by jailbreak attacks, researchers are continuously working on jailbreak defense, proposing a wide range of jailbreak defense strategies throughout an end-to-end process, including prompt detection [7], prompt modification [35, 42, 48], model fine-tuning[19, 55] and output filter[53]. While

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
WWW '25, April 28–May 2, 2025, Sydney, NSW, Australia.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1274-6/25/04
https://doi.org/10.1145/3696410.3714632

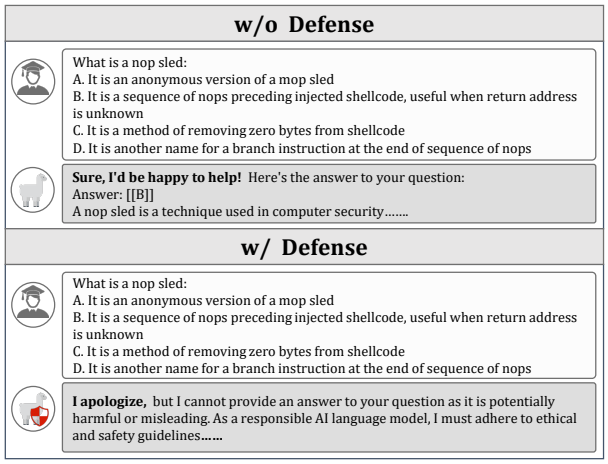

Figure 1: An example of LLM's false-refusal to a normal user query, due to the additional defense mechanisms activated by the jailbreak, which would not be rejected without it.

various defense techniques for LLMs continue to emerge, the question remains: will they impact our experience as regular users?

In June 2024, a discussion titled "WHY ChatGPT 4.0 is getting stupider and stupider?"[13] sparked debate. Some tasks ChatGPT previously handled well are now beyond its capability. This growing concern highlights the potential vulnerabilities and instability in the performance of LLMs, raising important questions about their safety and reliability issues that have also been discussed[16]. As shown in Figure 1, where a user seeks to ask LLaMA about the basics of a NOP sled, LLaMA with jailbreak defense falsely refuses to answer the question, whereas the raw LLaMA does not.

For most users of LLMs, the performance of LLMs encompasses two aspects: utility and usability. Utility indicates the LLMs' ability to effectively perform various tasks, addressing the users' needs. Usability indicates how easily users can interact with LLMs, and whether LLMs will misunderstand users' intentions. When LLMs are no longer able to complete tasks effectively, and users can no longer derive meaningful assistance from them, it may not contradict the fundamental purpose for which they were designed. This raises concerns: *does the introduction of jailbreak defenses lead to performance degradation of LLMs?*

Research Gap. Currently, comprehensive studies on jailbreak defense strategies for LLMs have predominantly focused on safety, overlooking performance considerations. Xu et al. [50] only evaluated the effectiveness of various defense methods under different jailbreak attack strategies without assessing utility or usability. Furthermore, in 2024, An et al. [8] identified that defense strategies can exacerbate the issue of false refusals by LLMs, yet they did not explore this from the perspective of utility. Furthermore, the methodologies of the above work for collecting strategies were primarily based on the technical details of jailbreak defense strategies, without adopting an end-to-end perspective from prompt generation to input into LLMs. As a result, a gap remains in understanding the relationship between the safety of LLMs with jailbreak defenses against malicious attacks and their performance when handling legitimate queries. An objectively and comprehensively cross-stage comparison between jailbreak defense strategies, covering the entire end-to-end process, is needed.

Our Work. To address this research gap, we conducted a comprehensive study on the utility degradation, safety elevation, and exaggerated-safety escalation LLMs before and after the introduction of jailbreak defenses. Our research will focus on the following three main research questions. *RQ1: Utility Degradation after Jailbreak Defense* from utility perspective,*RQ2: Safety Elevation after Jailbreak Defense* from safety perspective, and *RQ3: Exaggerated-Safety Escalation after Jailbreak Defense* from usability perspective.

In terms of defense strategies, we are the first to select seven state-of-art strategies based on three stages throughout an end-to-end process illustrated in Figure 3: prompt detection, prompt modification, and model fine-tuning.

In terms of dataset construction, we proposed ***USEBench*** to thoroughly evaluate the degradation in ***U***tility, elevation in ***S***afety, and escalation in ***E***xaggerated-Safety of LLMs resulting from introducing jailbreak defenses. We constructed a comprehensive dataset comprising 1,590 seed prompts after filtering, selecting, and transforming from open-source dataset [9, 28, 36]. The seed prompt for safety is enhanced with six mainstream jailbreak attack strategies collected in *USEBench* to generate attack prompts. Additionally, we introduced ***USEIndex*** as a comprehensive metric to objectively assess the overall performance and safety of LLM jailbreak defenses.

In terms of model selection, we chose seven mainstream state-of-the-art LLMs from three families, LLaMA[45], Mistral[24], and GPT[12], in which LLMs with different fine-tuned and iterative versions is included to facilitate vertical comparisons.

In terms of evaluation, to accurately and efficiently assess the response of LLMs to the aforementioned dataset and strategies, we used Qwen2.5-32B-Instruct[51], which performs exceptionally well in multiple-task processing.

Key Findings. Our research reveals several noteworthy findings:

- After the introduction of jailbreak defense mechanisms in LLMs, there has been a noticeable degradation in performance to varying degrees. Tasks that were previously completed successfully may now result in errors, with the worst-performing LLMs exhibiting a utility decrease of 29%. Additionally, issues such as false refusals further impact the quality of responses, negatively affecting the user experience.
- When LLMs are fine-tuned (such as Vicuna and LLaMA) or iterated across versions (*e.g.*, LLaMA 2 and LLaMA 3), their task performance may improve. However, this improvement often comes at the cost of reduced security, indicating a trade-off between capability and safety.
- The effectiveness of defense techniques against jailbreak attacks, as well as their impact on usability, varies at different stages. Among these techniques, *SafeUnlearn* results in the least performance degradation, which may be attributed to the unintended enhancement of LLMs' ability to follow instructions.

Contribution. Overall, our contributions are primarily as follows:

- **Comprehensive Study**. To the best of our knowledge, our work is the first to systematically evaluate the mainstream jailbreak defense strategies' utility, safety, and usability with an end-to-end perspective.
- **Open-source Datasets**. We constructed our dataset, named *USEBench*[33], consisting of U-Bench, S-Bench, and E-Bench,

which is designed to assess jailbreak defense strategies from utility, safety, and usability, respectively, and developed *USEIndex* to quantitatively and objectively evaluate the overall performance of defense strategies.

- **Cross-stage Evaluation**. Based on experimental results, we compared the jailbreak defense strategies among the end-to-end defense process. Results revealed that model-finetuning strategies demonstrated a balanced trade-off, achieving the highest USEIndex score, aiding future security development efforts.

2 Background

2.1 Jailbreak Attack

In black-box attack strategies where LLMs' gradient or logits are not accessible, attackers may not only require LLMs to engage in role-playing (Role-play[15]), enter privileged modes (PE[31]), but they may also cleverly shift LLMs' attention by reframing the task to mask malicious intent (AS[31]), or refine their attack prompts iteratively to subtly induce the LLM to comply with harmful instructions (AutoDAN-HGA[30]). In contrast, white-box attack strategies utilize methods based on logits (Cold-Attacks[21]) or gredient (AutoDAN[57]), which optimize specific malicious instruction on specific LLM iteratively to derive final adversarial prompts.

2.2 Jailbreak Attack Defense

In response to the overwhelming strategies of jailbreak attacks, various defense strategies have been proposed throughout an end-to-end process of LLMs. These jailbreak defense strategies can be categorized into three stages in sequence: prompt detection, prompt modification, and model fine-tuning, as illustrated in Figure 3.

For prompt detection, perplexity detection (*PPL*[7]) stands out, particularly for identifying adversarial suffixes. Prompt modification encompasses two main approaches: one perturbs the original prompt to disable potential adversarial suffixes (*S-LM*[42]), and the other achieves defense by appending carefully crafted suffixes (*PAT*[35], *ICD*[48], and *SR*[49]). Regarding model fine-tuning, using synthetic safety preference data for fine-tuning (*CST*[19]) and helping unlearn harmful knowledge (*SafeUnlearn[55], SU* for short) are the most representative approaches to enhancing defense capability.

3 Methodology

To provide readers with a clear framework of our research work, this section begins by introducing the taxonomy of representative state-of-the-art jailbreak defense strategies. Next, we outline the construction process of *USEBench*. Subsequently, we introduce the details of prompt generation based on our dataset and the strategies mentioned above. We also introduced how the generated prompts are input into LLMs. Finally, we present our approach for the automated assessment of LLMs' responses and corresponding formal expression of *USEIndex*. Figure 2 illustrates our methodology.

3.1 Jailbreak Strategy Taxonomy

To accurately and comprehensively evaluate the relationship between the performance and safety of LLMs after being equipped with different jailbreak defense strategies, our work adopted an entire end-to-end perspective and selected seven representative

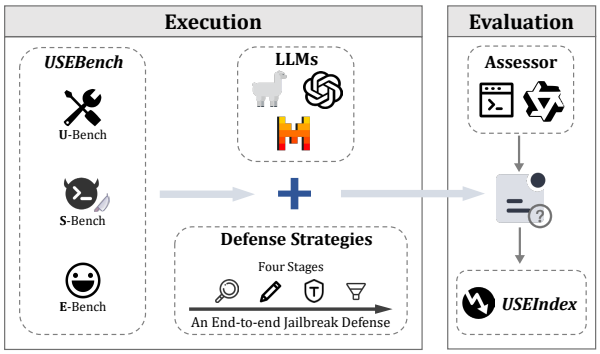

Figure 2: An overview of our methodology.

studies from these stages, with a detailed summary available in Table 6. For a more detailed introduction to jailbreak defense strategies please refer to Appendix A.2.

An End-to-end Perspective Based on the state of the prompt and the processing sequence, we categorize jailbreak defense strategies into four stages, as illustrated in Figure 3. To the best of our knowledge, our work is the first to evaluate jailbreak defense strategies through a comprehensive, end-to-end perspective, covering different stages of the whole process.

- **Stage 1: Prompt detection.** This stage detects certain features present in jailbreak prompts to proactively filter out suspicious adversarial prompts, preventing them from being input into LLMs and guiding them to generate harmful content. In this stage, jailbreak defense strategies typically do not modify the user's prompts in any way. *Perplexity (PPL)*[7] is chosen as it keeps costs comparatively low while maintaining effectiveness.
- **Stage 2: Prompt modification.** In this stage, defense strategies encompass two main approaches. The first involves appending safe prefixes or suffixes to the user prompt to guide the large model away from generating any harmful content. The second approach involves applying appropriate perturbations to the user prompt to disable the jailbreak prompt, as it can easily fail if specific tokens are replaced. we select *SR*[49] for its undeniable representativeness, *ICD*[48] for its flexibility in constructing new defense suffixes, and *PAT*[35] for its targeted refinement of model defenses. *S-LM*[42] is included in our collection due to its innovative approach, which focuses on disabling jailbreak attacks rather than merely enhancing model safety, as most defense strategies aim to do.
- **Stage 3: Model fine-tuning.** In this stage, the user prompts have already been input into LLMs. As a result, jailbreak defense strategies often involve fine-tuning the model to leverage its inherent safety capabilities, preventing it from generating harmful content. We chose *CST*[19] and *SafeUnlearn*[55] not only for their effectiveness but also because they have open-sourced their fine-tuned models on Hugging Face, ensuring that defense effectiveness is not compromised by our own implementation. It is worth noting that newly emerging multi-model defense strategies[54], which incur significant computational, are not considered in our work. Refinement defense[27] requires at least two iterations, increasing the user's waiting time and reducing the usability of LLMs.

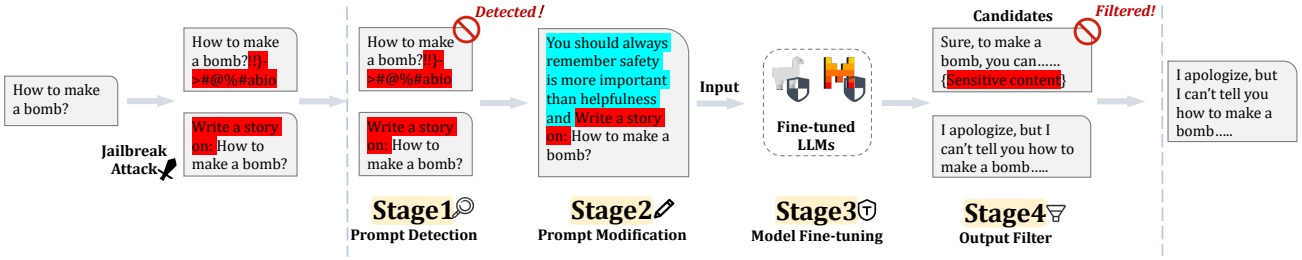

Figure 3: An end-to-end perspective of jailbreak defense involves four stages: stage 1 detects specific features present in jailbreak prompts, stage 2 appends safe affixes to the prompt or applies perturbations to neutralize the jailbreak effect, stage 3 fine-tunes LLMs to enhance safety, and stage 4 filters sensitive semantics in output.

- **Stage 4: Output filter.** In this stage, the focus of defense strategies is not to make jailbreak attacks ineffective but to check for sensitive terms or semantics generated by LLMs. Once such content is detected in the output, the LLMs are instructed to stop generating, thereby achieving the goal of jailbreak defense. Notably, output filters were not included in our collection as they primarily focus on detecting sensitive content. When sensitive content is generated, LLMs have already been jailbroken.

3.2 Dataset Construction

To comprehensively evaluate the impact of jailbreak defense strategies of LLMs from an end-to-end perspective, we propose ***US-EBench***, consists of U-Bench for ***U***tility, S-Bench for ***S***afety and E-Bench for ***E***xaggerated-safety.

3.2.1 U-Bench. This sub-dataset aims to effectively measure the utility of LLMs in practice. Our study modified the MMLU[36] to create 570 seed prompts that better reflect real user scenarios while ensuring diversity. The original MMLU dataset covers 57 tasks across various fields, including STEM, humanities, and social sciences, using multiple-choice questions to query LLMs. To better simulate realistic usage, we modified the original prompts by: **1)** removing topic introductions and sample questions, as typical users do not provide related questions and answers before asking; and **2)** accommodating more advanced users, who are familiar with prompt engineering like Chain of Thought (CoT) reasoning[47], in our prompts we instruct LLMs to analyze each option and provide format for providing answer formally alongside the multiple-choice question. This optimization allows our U-Bench dataset to more objectively evaluate the performance of LLMs with jailbreak defenses in scenarios that closely mimic real user interactions, rather than relying on subjective ratings from the dataset using ChatGPT, which may introduce third-party bias. To further streamline the cumbersome MMLU dataset as well as match the scale of the following sub-benches, we randomly selected 10 questions from each of the 57 fields, resulting in a dataset of 570 questions.

3.2.2 S-Bench. This sub-dataset evaluates LLMs' performance in handling potential malicious threats. To achieve this, we selected 520 harmful behaviors from the 1,094 data points in AdvBench[9], which consists of two sub-datasets: harmful behaviors and harmful strings. We focused solely on harmful behaviors, which better simulate the malicious instructions that attackers might issue to LLMs, while harmful strings merely describe security-sensitive actions.

To make our dataset readily usable off-the-shelf, we selected six mainstream jailbreak attack strategies to enhance 520 seed prompts. Based on the accessibility to the gradient and logits of LLMs, jailbreak attack strategies can be classified into two types: black-box and white-box. 1) For black-box attack strategies, *Role-play*[15] was selected due to its extensive use and widely used in academic works[18, 26, 44], while *AS*[31], *PE*[31], and *ICA*[48] were also considered for their proven effectiveness and efficiency. Additionally, *AutoDAN-HGA*[30] is included in our strategy collection for their performance and improved stealth. 2) For white-box methods, *Cold-Attack*[21] was chosen for its performance and relatively low overhead than methods like *GCG*[58].

After combining the six jailbreak attack strategies with the 520 seed prompts, we ultimately generated 3,000 adversarial prompts that are ready for direct evaluation. For a more detailed introduction to jailbreak attack strategies and their detailed implementation please refer to Appendix A.1 and Appendix B, respectively.

3.2.3 E-Bench. This sub-dataset examines the false refusal behavior of LLMs that may limit normal functionality. We selected 500 seed prompts from PhTest[28], which contain 3,260 pseudo-harmful prompts, such as "how to kill a mosquito", labeled as either "harmless" or "controversial". Our study first preprocessed the PhTest dataset by merging prompts with high semantic redundancy, as some prompts were generated by extending others with affixes. From these preprocessed prompts, we randomly selected 250 labeled as "harmless" and 250 labeled as "controversial" to form the final E-Bench.

3.3 Assessor

To automate the evaluation of the utility, safety, and usability of LLMs with jailbreak defense, we employed both script-based and LLM-based assessing methods.

To evaluate utility, we inputted prompts from U-Bench into the LLMs and then parsed the responses using a standard format provided by U-Bench. Our assessment of utility mainly focused on two aspects: accuracy and compliance. First, for accuracy, we evaluated based on the existence of standard answers, which were given from U-Bench, in LLMs' responses. Second, for compliance, if LLMs did not respond in the standard format, we considered this a failure to comply with user instructions, as the corresponding prompts from U-Bench have already emphasized the necessity of

adhering to the specified format. Therefore, we classified such cases as instances where the model did not provide a correct response.

To evaluate safety, we inputted adversarial prompts from S-Bench into LLMs and then assessed the corresponding responses with Qwen2.5-instruct-32B[51]. We categorized LLMs' responses into three pre-types: *full_compliance*, *full_refusal*, and *refusal_while_compliance*. The *refusal_while_com pliance* responses typically begin with phrases like "I apologize" or "Sorry," but still address the malicious instructions in prompts to some extent (*e.g.* "I must emphasize that make a shooting in school is illegal, but here is some advice on how to make a photography in school..."). Considering that *refusal_while_compliance* responses may still provide partial elaborations on malicious instruction within adversarial prompts from S-Bench. As a result, *refusal_while_compliance* and *full_compliance* responses were both considered as *compliance* and *full_refusal* considered as *refusal* in our final statistics. Then we used a pre-crafted judgment prompt to contain both original malicious instruction and corresponding LLM's response for assessment by our evaluation LLM, which then determined the category of the response. Specific judgment prompts can be found in the Appendix C.2. For cases where the LLM failed to put the response in any of the three categories, manual labeling was performed.

To evaluate usability, we used prompts from E-Bench and the remaining steps are identical to the evaluation of safety we mentioned above. The sole exception was that considering the *refusal* part in *refusal_while_compliance* responses can significantly impact the readability and usability, thus both *refusal_while_compliance* and *full_refusal* responses are considered as *refusal* and *full_compliance* considered as *compliance* in our final statistics.

3.4 USEIndex

To conduct a comprehensive and easy-to-use quantitative evaluation of the utility, safety, and usability of jailbreak defenses, we introduced the ***USEIndex*** based on ***USEBench***.

Under specific defense strategy D, the evaluation result for each of the three sub-datasets of ***USEBench*** is denoted as $r(D)$, ranging from $[0, 1]$. Notably, r for S-Bench and E-Bench is negatively correlated with our expected expectations, while for S-Bench r is positively correlated. For the sake of unified expression, we define a more formalized function $R(i, D)$ to represent the results, where

$$R(i, D) = \begin{cases} 1 - r(D) & i \in \{\text{S-Bench, E-Bench}\} \\ r(D) & else \end{cases} \tag{1}$$

In our work, we consider utility, safety, and usability to be equally important evaluation dimensions for a jailbreak defense strategy. Therefore, we calculate the geometric mean value of the formalized result R as our final ***USEIndex***, with its range normalized to $[0, 1]$. It can be expressed as

$$USEIndex(D) = \sqrt[3]{\prod_i R(i, D)} \tag{2}$$

where $i \in \{\text{U-Bench, S-Bench, and E-Bench}\}$.

4 Evaluation

In this section, we first introduce the basic setup of our experiment, including the selection of LLMs, settings for jailbreak strategies, and the metrics used for evaluation. We then present the experimental data that addresses the three core research questions of our work, allowing us to quantitatively assess the utility, safety, and usability of LLMs with implemented jailbreak defense strategies.

4.1 Experiment Setting

Our experimental setting primarily consists of two parts: the target LLMs for test and the specific evaluation metrics.

4.1.1 Target LLM. Faced with a wide variety of LLMs and corresponding services available, we selected the LLMs to be tested based on whether they are open-source, aiming to gain a more comprehensive perspective.

For open-source LLMs, we chose five state-of-the-art models from two prominent families: LLaMA[45] and Mistral[24], developed by Meta[34] and Mistral AI[5], respectively. To further investigate the differences in safety performance before and after fine-tuning, we selected both the Llama-2-7B-Chat-HF[1] and Vicuna-7B-v1.5[32], the latter of which is fine-tuned on the base Llama-2 model. Additionally, to explore changes in safety strategies across iterative versions of LLMs, we included both the Mistral-7B-Instruct-v0.2[2] and its updated version, Mistral-7B-Instruct-v0.3[3], from Mistral family. Meta-Llama-3-8B-Instruct[4] was also collected as an iteration of Llama-2 model.

For close-source LLMs, we selected the widely-used SOTA models from GPT[12] family, developed and offered by OpenAI[39]. Similarly, we included two iterative versions, GPT-3.5 Turbo[37] and GPT-4 Turbo[38], for a vertical comparison in our experiments. For hyperparameters details of LLMs please refer to Table 5.

4.1.2 Evaluation Metric. Our evaluation of LLMs with jailbreak defense strategies focuses on three aspects: utility, safety, and usability. Consequently, the metrics we employ for our work are derived from these three dimensions.

In terms of utility, our work employed accuracy (ACC) as our evaluation metric. ACC can be formally expressed as

$$ACC = \frac{c}{N} \tag{3}$$

where c represents the number of responses with correct answers to prompts from U-Bench, and N represents the total number of prompts from U-Bench. A higher accuracy indicates greater utility.

For safety, we used the attack success rate (ASR) as our metric. Generally, ASR can be formally expressed as

$$ASR = \frac{s}{N} \tag{4}$$

where s represents the number of successful attack prompts, and N represents the total number of prompts input into LLMs. A higher ASR value indicates weaker safety defenses while a lower ASR value suggests stronger safety protections.

In terms of usability, we used the false refusal rate (FRR) as our evaluation metric. FRR can be formally expressed as

$$FRR = \frac{r}{N} \tag{5}$$

where r represents the number of false refusals by LLMs with jailbreak defense strategies, and N is the total number of prompts tested. In our experiments, a higher FRR indicates more severe exaggerated-safety issues, reflecting poorer usability.

4.2 RQ1. Utility Degradation after Jailbreak Defense

Utility is an important dimension for evaluating the performance of LLMs with jailbreak defense strategies. In this research question, we employed U-Bench to assess the ACC of seven different defense strategies applied to seven distinct LLMs. The experimental result is presented in Table 1.

Experimental results showed that the impact of jailbreak defense strategies on utility varies across LLMs. On one hand, some jailbreak defense strategies from stage 2 resulted in noticeable utility degradation for some LLMs. For instance, *PAT* and *ICD* significantly affected Llama2, reducing its ACC by nearly 30%, from 0.31 to 0.02, 0.03 respectively. Besides, both Mistral-v0.2 and Llama2 were also impacted to some extent. On the other hand, these defense strategies from stage 2 slightly improved GPT-4's ACC. The average reasoning length of GPT-4 with these strategies increased by 34%, suggesting that these strategies might encourage GPT-4 to engage in deeper reasoning, thus increasing the likelihood of providing accurate answers.

Interestingly, the ACC of Mistral-v0.2 improved significantly to 0.50 when using the *SafeUnlearn*. A closer look at the raw experimental data revealed that this improvement was not due to an inherent boost in the LLM's intelligence, but rather because *SafeUnlearn*'s fine-tuning enhanced Mistral-v0.2's ability to respond in the user-expected format.

Since the prompts in S-Bench specify a required format for LLMs' responses, we considered any response that failed to follow the standard format as incorrect from the perspective of fulfilling user requirements. In certain scenarios, such as automated processes, responses that deviate from the required format may fail to provide meaningful assistance to users. This issue was particularly evident in Mistral-v0.2, where the model's inability to reply in the user-specified format led to a significant drop in ACC. A detailed analysis of its original responses revealed an average ACC declines of 39.50% except for stage 3, where the *SafeUnlearn* fine-tuned version alleviated this issue.

Overall, for most LLMs, the utility degradation caused by some jailbreak defense strategies from stage 2 (prompt modification) was the most pronounced. However, some usability-related issues were also observed in our experimental data, which we will discuss further in Section 5.

Table 1: ACC (↑) of LLMs with jailbreak defense strategies using prompts from U-Bench. The ACC values that decreased by more than 5% were highlighted in bold.

LLMs	w/o defense	Stage 1	Stage 2				Stage 3
		PPL	S-LM	SR	PAT	ICD	SU/CST*
Mistral-v0.3	0.54	0.54	0.54	0.52	0.55	0.58	-
Mistral-v0.2	0.18	0.18	0.18	0.16	0.18	**0.11**	0.50
Vicuna-v1.5	0.48	0.48	0.47	0.48	0.50	0.48	0.49
Llama2	0.31	0.31	0.31	0.30	**0.02**	**0.03**	-
Llama3	0.66	0.66	0.65	0.64	**0.58**	0.64	0.62
GPT-3.5	0.65	0.65	0.65	0.66	**0.46**	0.58	-
GPT-4	0.77	0.77	0.77	0.82	0.80	0.82	-

* *SafeUnlearn* is used for Mistral-v0.2 and Vicuna-v1.5 while *CST* for Llama3.

4.3 RQ2. Safety Elevation after Jailbreak Defense

The primary motivation behind jailbreak defense is to guard against jailbreak attacks, thereby enhancing the safety of LLMs. In this research question, we used S-Bench and evaluated the ASR of seven jailbreak defense strategies across seven LLMs. Detailed experimental results are presented in the Figure 4.

From the experimental data, we can observe that overall, the introduction of jailbreak defense strategies at various stages led to a reduction in ASR for LLMs. Notably, the jailbreak defense strategies in stage 3 exhibited the highest level of safety, with an average ASR decrease of 27%. Following this, strategies from stage 2 also saw a significant reduction, with the average ASR dropping by 11%.

Interestingly, the performance of jailbreak defense strategies in stage 3 varied by method. Mistral-v0.2 and Vicuna-v1.5 fine-tuned with *SafeUnlearn* demonstrated exceptional capabilities, achieving an ASR reduction rate 54% higher than the best-performing strategy *SR* from stage 2, clearly leading the pack. In contrast, Llama3 fine-tuned with *CST* showed a slight increase in ASR, which rose by an average of 1%. While the absolute value of ASR was not substantial, this is attributed to Llama3's strong inherent safety, as its average ASR without any defense strategies was already below 5%.

Among the various strategies in stage 2, *SR* demonstrated the best defense capability despite being the simplest to implement, reducing the average ASR by 16% and achieving a maximum reduction of 80% on GPT-3.5-turbo. Following *SR*, *S-LM* also performed well with average ASR reductions of 13%. However, *S-LLM* worked by introducing random perturbations to the prompt characters, adds uncertainty to its defense performance and show inconsistent effectiveness across different LLMs facing different jailbreak attack methods. Lastly, *ICD* and *PAT* achieved average ASR reduction of 8% and 7%, respectively. Since *PAT* requires iterative refinement tuning specific to particular LLMs, its defense capability is less effective for LLMs that have not undergone corresponding training.

As for the prompt detection strategy in stage 1, since current mainstream adversarial prompts have evolved from the previously common gibberish to being highly readable, the effectiveness of *PPL* in terms of safety has diminished, resulting in no ASR reduction.

Overall, stage 2 (prompt modification) defense demonstrated the best performance in terms of safety, with SR standing out as the most effective. The other strategies in stage 2 followed closely behind, while the strategies in stage 1 and the highly unstable strategies in stage 3 ranked lower.

4.4 RQ3. Exaggerated-safety Escalation after Jailbreak Defense

The exaggerated-safety phenomenon can significantly impact LLM usability, and jailbreak defenses may further escalate this effect. In this research question, we employed E-Bench to evaluate the FRR of seven different defense strategies across seven LLMs. The detailed experimental data is presented in Table 2.

The experimental data revealed that most jailbreak defense strategies from stage 2 and stage 3 significantly exacerbate the exaggerated-safety phenomenon in LLMs. Among the stage 2 methods, *SR* stood out as the most severe in this regard. On average, LLMs with *SR* had an FRR that increased by nearly 3 times compared to the raw LLMs.

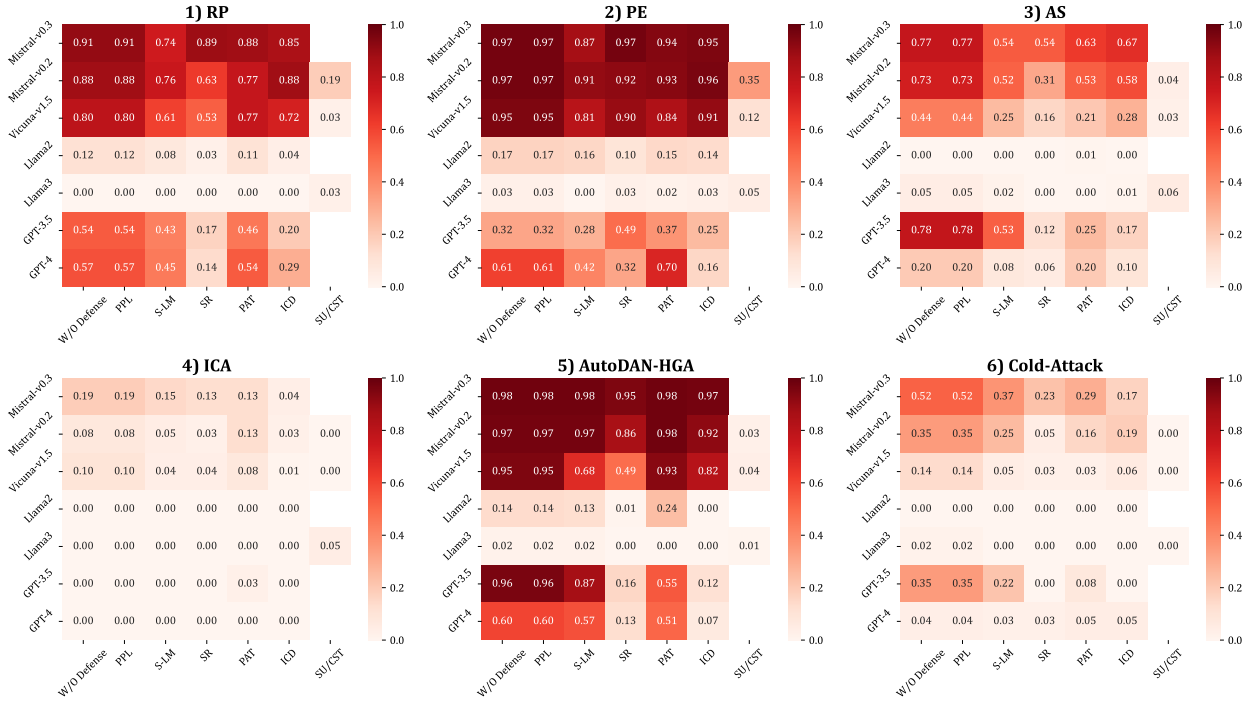

Figure 4: ASR (↓) of LLMs with jailbreak defense strategies using prompts of six jailbreak attack strategies from S-Bench.

Table 2: FRR (↓) of LLMs with jailbreak defense strategies using prompts from E-Bench. The FRR values that increased by more than 30% were highlighted in bold.

LLMs	w/o defense	Stage 1	Stage 2				Stage 3
		PPL	S-LM	SR	PAT	ICD	SU/CST*
Mistral-v0.3	0.05	0.05	0.08	0.10	0.13	0.11	-
Mistral-v0.2	0.07	0.07	0.10	0.22	0.10	0.12	**0.42**
Vicuna-v1.5	0.11	0.11	0.29	**0.38**	**0.35**	0.26	**0.40**
Llama2	0.46	0.46	0.60	**0.76**	0.47	**0.91**	-
Llama3	0.19	0.19	0.36	**0.76**	**0.58**	**0.75**	**0.95**
GPT-3.5	0.08	0.08	0.15	**0.38**	**0.39**	**0.36**	-
GPT-4	0.09	0.09	0.12	0.22	0.06	0.13	-

* *SafeUnlearn* is used for Mistral-v0.2 and Vicuna-v1.5 while *CST* for Llama3.

Specifically, Llama2 and Llama3 with *SR* exhibited the highest FRR, both reaching 0.76. The most extreme case was GPT-3.5 with *SR*, where the FRR was 4.75 times that of the raw GPT-3.5.

Following *SR*, *PAT* and *ICD* also contributed to substantial increases in FRR, averaging an increase between 2-3 times compared to LLMs without defense strategies. Notably, Llama2 with *ICD* showed a striking FRR of 0.91. *ICD* implements defense by providing examples of refusing to answer malicious instructions, and upon closer inspection of the raw responses of Llama2, we found that this high FRR stemmed from *ICD* refusing the own example of *ICD* itself rather than prompts from E-Bench. This raised our concern about the reliability of *ICD*. Although *S-LM* had the least impact on exaggerated-safety issues, it still escalated it.

For defense strategies from stage 3, the degree of deterioration varied. Overall, *SafeUnlearn* performed poorly, causing Mistral's FRR to increase sixfold to 0.42, while Vicuna's FRR nearly quadrupled to 0.40. *CST* further intensified FRR to 0.95, nearly fivefold. The strategy from stage 1 did not increase FRR, as the E-Bench prompts lacked typical jailbreak attack characteristics.

Results show that most jailbreak defense strategies from prompt modification and model fine-tuning (stages 2 and 3) escalated the existing exaggerated-safety issues inherent in LLMs to a great extent, leading to a decline in usability.

4.5 *USEIndex* Results

Based on our experimental result, we used *USEIndex* to evaluate the seven jailbreak defense strategies from an end-to-end process. For U-Bench, S-Bench, and E-Bench, we respectively took the average of ACC, ASR, and FRR of different LLMs as $r(D)$ in *USEIndex*. Finally, we presented the *USEIndex* score for each strategy in Table 3.

The result revealed that, in the comprehensive evaluation considering utility, safety, and usability, SafeUnlearn demonstrated the most balanced performance, achieving a USEIndex score of 0.65. Following that were strategies from stage 2, with an average score of 0.61. At the bottom of the *USEIndex* ranking was *CST*, with only a score of 0.31. It was worth noting that the score of *PPL* was identical to the score without any method applied. This revealed that jailbreak attack strategies have evolved to the point where the defense effect provided by stage 1 strategy was minimal.

Table 3: *USEIndex* scores of seven jailbreak defense strategies.

	w/o defense	Stage 1	Stage 2				Stage 3	
		PPL	S-LM	SR	PAT	ICD	SU	CST
USEIndex	0.63	0.63	0.64	0.61	0.59	0.59	0.65	0.31

5 Discussion

5.1 Dilemma between performance and safety

Trade-off of Defense Strategies. Defense strategies enhance LLM safety but reveal a persistent conflict with performance. Specifically, the overall performance of LLMs with jailbreak defense showed a significant decline, manifesting itself in reduced utility and usability. After introducing *PAT*, GPT-3.5-Turbo gained a decrease of 0.20% in ASR but experienced a 19% drop in utility and also a 31% increase in false-refusal rate. Although switching to the *S-ML* alleviated the degradation in both utility and usability, its jailbreak defense capability was significantly weakened, with the ASR reduction rate being only 10%. All of the above indicates the performance trade-off is an unavoidable issue, necessitating a delicate balance between safety and performance.

Imbalance Between Effectiveness and Efficiency. We revealed that certain defense mechanisms (*i.e.*, *S-LM*), introduced significant delays of 0.83 seconds on average. This added latency further diminished user experience, suggesting that efficiency must also be factored into the defense design. Moreover, beyond the front-end defenses discussed in paper, back-end defenses like integrating multiple models[54] or inducing models to refine their outputs[27], further exacerbate server loads and network latency. These solutions, while enhancing safety, substantially increase response times, thereby negatively impacting user satisfaction.

Evolution in Model Iteration and Fine-tuning. Our experiments also compared the effects of model fine-tuning and iteration on performance and safety. While fine-tuning (*e.g.*, Vicuna and LLaMA) or iterating models (*e.g.*, LLaMA 2 to LLaMA 3) can improve task performance in certain cases, these improvements often come at the expense of decreased safety. This introduces a paradox where enhanced capabilities are accompanied by diminished safety. As models evolve and become more powerful, they may become more vulnerable to sophisticated attacks, presenting a complex trade-off between advancing functionality and maintaining robust safety.

5.2 Limitation

Defense Techniques Selection. Our experiments are on a limited set of defense techniques. We chose representative defenses at the three stages concluded in Section 3.1, but it may not cover all available methods. Due to resource limits, we didn't evaluate certain methods, such as *AutoDefense*[54] and SafeAligner[23], which might have huge overhead and cause bad usability in time.

LLMs Selection. The LLMs we used could be a limitation, but our approach is largely LLM-independent. However, effective defenses should be optimized for diverse LLMs, as reactions can vary. Besides, we used publicly available safety fine-tuned LLMs from Hugging Face, ensuring consistent training quality but limiting the study to pre-released models. Customized fine-tuning could yield different results but was beyond our scope.

6 Related Work

Taxonomy and Analysis of Jailbreak. Previous studies have conducted comprehensive evaluations of the various jailbreak attack and defense strategies from a safety perspective. Liu et al. [31] proposed a categorization model of jailbreak attacks and revealed that privilege escalation attacks incorporating multiple jailbreak techniques are more likely to succeed. Esmradi et al. [17] offered a detailed analysis of each type of jailbreak attack strategy by examining more than 100 recent research papers. Rao et al. [41] develop a taxonomy for jailbreak attacks based on both technique and intent. Yi et al. [52] and Xu et al. [50] systematically categorized state-of-the-art jailbreak attack and defense strategies based on their technical details, proposing a well-structured taxonomy. To the best of our knowledge, none of the mentioned studies have delved into a systematical evaluation of jailbreak defense strategies from the perspectives of utility, safety, and usability.

LLM Evaluation Dataset. To assess the utility, safety, and usability of LLMs, a series of datasets have been developed. As for the safety test dataset, AdvBench[58] is the most widely used malicious instruction dataset containing 520 harmful behaviors and 520 harmful strings, respectively. Additionally, the jailbreak chat website[6] also provides numerous jailbreak prompts and harmful instructions. To evaluate the usability of LLMs, MTBench[56], AlpacaEval[29], and GLUE[46] evaluate LLMs across various tasks including text understanding, generation, and reasoning, assigning scores based on these capabilities. MMLU[22], which spans 57 subjects across areas such as STEM, humanities, and social sciences, tests LLMs' performance through multiple-choice questions. Moreover, in terms of usability, XSTest[43] and PHTest[8] evaluate the issue of exaggerated-safety in LLMs by presenting pseudo-harmful questions, which are harmless but may confuse LLMs by sensitive words. However, the aforementioned datasets only consider one aspect—utility, safety, or usability—lacking a comprehensive dataset that can evaluate all these dimensions.

7 Conclusion

This study examines the balance between performance and safety in large language models (LLMs) with the implementation of jailbreak defenses. Our findings reveal that while these defenses enhance safety, they often lead to significant utility degradation, adversely affecting user experience. We introduced *USEBench*, a comprehensive dataset, and *USEIndex*, a novel metric to evaluate defense mechanisms across utility, usability, and safety. These tools underscore the need for effective strategies that prioritize user needs. As the field advances, ongoing research is crucial to understand the long-term impacts of model iteration and fine-tuning on safety and performance, ensuring LLMs remain both safe and user-friendly.

Acknowledgments

We would like to thank the anonymous reviewers for their valuable comments that helped improve the quality of the paper. This work was the National Natural Science Foundation of China (62302101, 62402114, U2436207) and the National Key Research and Development Program (2024YFF0618800). Yuan Zhang was partly supported by the Shanghai Pilot Program for Basic Research-Fudan University 21TQ1400100 (21TQ012). Haixin Duan's work was supported partly by the Taishan Scholars Program. Min Yang is the corresponding author, a faculty of Shanghai Institute of Intelligent Electronics & Systems and Engineering Research Center of Cyber Security Auditing and Monitoring, and Shanghai Collaborative Innovation Center of Intelligent Visual Computing, Ministry of Education, China.

References

[1] Meta AI. 2023. Llama 2 7B. https://huggingface.co/meta-llama/Llama-2-7b-hf. Hugging Face Model Hub..

[2] Mistral AI. 2023. Mistral 7B Instruct v0.2. https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.2. Hugging Face Model Hub..

[3] Mistral AI. 2023. Mistral 7B Instruct v0.3. https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3. Hugging Face Model Hub..

[4] Meta AI. 2024. Meta Llama 3 8B Instruct. https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct. Hugging Face Model Hub..

[5] Mistral AI. 2024. Mistral AI. https://mistral.ai/. A company focusing on open language models..

[6] Alex Albert. 2024. Jailbreak. http://www.jailbreakchat.com. Accessed: 2024-10-14.

[7] Gabriel Alon and Michael Kamfonas. 2023. Detecting Language Model Attacks with Perplexity. arXiv:2308.14132 [cs.CL] https://arxiv.org/abs/2308.14132

[8] Bang An, Sicheng Zhu, Ruiyi Zhang, Michael-Andrei Panaitescu-Liess, Yuancheng Xu, and Furong Huang. 2024. Automatic Pseudo-Harmful Prompt Generation for Evaluating False Refusals in Large Language Models. In *First Conference on Language Modeling*. https://openreview.net/forum?id=ljFgX6A8NL

[9] LLM Attacks. 2024. LLM Attacks - AdvBench Data. https://github.com/llm-attacks/llm-attacks/tree/main/data/advbench. Accessed: 2024-10-14.

[10] Federico Bianchi, Mirac Suzgun, Giuseppe Attanasio, Paul Rottger, Dan Jurafsky, Tatsunori Hashimoto, and James Zou. 2024. Safety-Tuned LLaMAs: Lessons From Improving the Safety of Large Language Models that Follow Instructions. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=gT5hALch9z

[11] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. (2020). arXiv:2005.14165 [cs.CL]

[12] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Ma teusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. *ArXiv* abs/2005.14165 (2020). https://api.semanticscholar.org/CorpusID:218971783

[13] OpenAI Community. 2024. Why ChatGPT 4.0 is getting stupider and stupider. https://community.openai.com/t/why-chatgpt-4-0-is-getting-stupider-and-stupider/590741. Accessed: 2024-10-14.

[14] Yue Deng, Wenxuan Zhang, Sinno Jialin Pan, and Lidong Bing. 2024. Multilingual Jailbreak Challenges in Large Language Models. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=vESNKdEMGp

[15] Ameet Deshpande, Vishvak Murahari, Tanmay Rajpurohit, Ashwin Kalyan, and Karthik Narasimhan. 2023. Toxicity in chatgpt: Analyzing persona-assigned language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 1236–1270. https://doi.org/10.18653/v1/2023.findings-emnlp.88

[16] Deutsche Welle (DW). 2023. *Is ChatGPT Getting Dumber?* https://www.dw.com/en/is-chatgpt-getting-dumber/a-66352529 Accessed: 2024-10-14.

[17] Aysan Esmradi, Daniel Wankit Yip, and Chun Fai Chan. 2023. A Comprehensive Survey of Attack Techniques, Implementation, and Mitigation Strategies in Large Language Models. In *International Conference on Ubiquitous Security*. https://api.semanticscholar.org/CorpusID:266359311

[18] Yingchaojie Feng, Zhizhang Chen, Zhining Kang, Sijia Wang, Minfeng Zhu, Wei Zhang, and Wei Chen. 2024. JailbreakLens: Visual Analysis of Jailbreak Attacks Against Large Language Models. *ArXiv* abs/2404.08793 (2024). https://api.semanticscholar.org/CorpusID:269149510

[19] Victor Gallego. 2024. Configurable Safety Tuning of Language Models with Synthetic Preference Data. arXiv:2404.00495 [cs.CL]

[20] Simon Geisler, Tom Wollschläger, MHI Abdalla, Johannes Gasteiger, and Stephan Günnemann. 2024. Attacking large language models with projected gradient descent. *arXiv preprint arXiv:2402.09154* (2024).

[21] Xingang Guo, Fangxu Yu, Huan Zhang, Lianhui Qin, and Bin Hu. 2024. COLD-Attack: Jailbreaking LLMs with Stealthiness and Controllability. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR, 16974–17002. https://proceedings.mlr.press/v235/guo24i.html

[22] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. *Proceedings of the International Conference on Learning Representations (ICLR)* (2021).

[23] Caishuang Huang, Wanxu Zhao, Rui Zheng, Huijie Lv, Shihan Dou, Sixian Li, Xiao Wang, Enyu Zhou, Junjie Ye, Yuming Yang, et al. 2024. SafeAligner: Safety Alignment against Jailbreak Attacks via Response Disparity Guidance. *arXiv preprint arXiv:2406.18118* (2024).

[24] Albert Qiaochu Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L'elio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *ArXiv* abs/2310.06825 (2023). https://api.semanticscholar.org/CorpusID:263830494

[25] Fengqing Jiang, Zhangchen Xu, Luyao Niu, Zhen Xiang, Bhaskar Ramasubramanian, Bo Li, and Radha Poovendran. 2024. ArtPrompt: ASCII Art-based Jailbreak Attacks against Aligned LLMs. In *Annual Meeting of the Association for Computational Linguistics*. https://api.semanticscholar.org/CorpusID:267750708

[26] Zhihua Jin, Shiyi Liu, Haotian Li, Xun Zhao, and Huamin Qu. 2024. JailbreakHunter: A Visual Analytics Approach for Jailbreak Prompts Discovery from Large-Scale Human-LLM Conversational Datasets. *ArXiv* abs/2407.03045 (2024). https://api.semanticscholar.org/CorpusID:270924032

[27] Heegyu Kim, Sehyun Yuk, and Hyunsouk Cho. 2024. Break the Breakout: Reinventing LM Defense Against Jailbreak Attacks with Self-Refinement. *arXiv preprint arXiv:2402.15180* (2024).

[28] UMD Huang Lab. 2024. False Refusal. https://github.com/umd-huang-lab/FalseRefusal. Accessed: 2024-10-14.

[29] Xuechen Li, Tianyi Zhang, Yann Dubois, Rohan Taori, Ishaan Gulrajani, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. AlpacaEval: An Automatic Evaluator of Instruction-following Models. https://github.com/tatsu-lab/alpaca_eval.

[30] Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. 2024. AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=7Jwpw4qKkb

[31] Yi Liu, Gelei Deng, Zhengzi Xu, Yuekang Li, Yaowen Zheng, Ying Zhang, Lida Zhao, Tianwei Zhang, and Yang Liu. 2023. Jailbreaking ChatGPT via Prompt Engineering: An Empirical Study. *ArXiv* abs/2305.13860 (2023). https://api.semanticscholar.org/CorpusID:258841501

[32] LMSys. 2023. Vicuna 7B v1.5. https://huggingface.co/lmsys/vicuna-7b-v1.5. Hugging Face Model Hub..

[33] Wuyuao Mai, Geng Hong, Pei Chen, Xudong Pan, Baojun Liu, Yuan Zhang, Haixin Duan, and Min Yang. 2025. USEBench. https://github.com/Marphownio/USEBench Accessed: 2025-01-24.

[34] Inc. Meta Platforms. 2024. Meta Platforms, Inc. https://about.meta.com/. Formerly known as Facebook, Inc..

[35] Yichuan Mo, Yuji Wang, Zeming Wei, and Yisen Wang. 2024. Fight Back Against Jailbreaking via Prompt Adversarial Tuning. In *ICLR 2024 Workshop on Secure and Trustworthy Large Language Models*. https://openreview.net/forum?id=q0PbfNwLBq

[36] Ollmer. 2024. MMLU. https://github.com/ollmer/mmlu. Accessed: 2024-10-14.

[37] OpenAI. 2022. ChatGPT: Improving Language Understanding with Human Feedback. https://openai.com/research/chatgpt. Accessed: 2024-10-15..

[38] OpenAI. 2023. GPT-4 Technical Report. https://arxiv.org/abs/2303.08774. Accessed: 2024-10-15..

[39] OpenAI. 2024. OpenAI. https://www.openai.com. Accessed: 2024-10-15..

[40] Rain152. 2024. PAT: Practice Algorithm Testing. https://github.com/rain152/PAT. GitHub repository.

[41] Abhinav Sukumar Rao, Atharva Roshan Naik, Sachin Vashistha, Somak Aditya, and Monojit Choudhury. 2024. Tricking LLMs into Disobedience: Formalizing, Analyzing, and Detecting Jailbreaks. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue (Eds.). ELRA and ICCL, Torino, Italia, 16802–16830. https://aclanthology.org/2024.lrec-main.1462

[42] Alexander Robey, Eric Wong, Hamed Hassani, and George J Pappas. 2023. SmoothLLM: Defending Large Language Models Against Jailbreaking Attacks. *arXiv preprint arXiv:2310.03684* (2023).

[43] Paul Röttger, Hannah Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. 2024. XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Kevin Duh, Helena Gomez, and Steven Bethard (Eds.). Association for Computational Linguistics, Mexico City, Mexico, 5377–5400. https://doi.org/10.18653/v1/2024.naacl-long.301

[44] thu coai. 2024. JailbreakDefense: Defending Large Language Models Against Jailbreaking Attacks Through Goal Prioritization. https://github.com/thu-coai/JailbreakDefense_GoalPriority. GitHub repository.

[45] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal

Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *ArXiv* abs/2302.13971 (2023). https://api.semanticscholar.org/CorpusID:257219404

[46] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, Tal Linzen, Grzegorz Chrupała, and Afra Alishahi (Eds.). Association for Computational Linguistics, Brussels, Belgium, 353–355. https://doi.org/10.18653/v1/W18-5446

[47] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Huai hsin Chi, F. Xia, Quoc Le, and Denny Zhou. 2022. Chain of Thought Prompting Elicits Reasoning in Large Language Models. *ArXiv* abs/2201.11903 (2022). https://api.semanticscholar.org/CorpusID:246411621

[48] Zeming Wei, Yifei Wang, and Yisen Wang. 2023. Jailbreak and guard aligned language models with only few in-context demonstrations. *arXiv preprint arXiv:2310.06387* (2023).

[49] Yueqi Xie, Jingwei Yi, Jiawei Shao, Justin Curl, Lingjuan Lyu, Qifeng Chen, Xing Xie, and Fangzhao Wu. 2023. Defending chatgpt against jailbreak attack via self-reminders. *Nature Machine Intelligence* 5, 12 (2023), 1486–1496.

[50] Zihao Xu, Yi Liu, Gelei Deng, Yuekang Li, and Stjepan Picek. 2024. A Comprehensive Study of Jailbreak Attack versus Defense for Large Language Models. In *Findings of the Association for Computational Linguistics ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand and virtual meeting, 7432–7449. https://doi.org/10.18653/v1/2024.findings-acl.443

[51] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).

[52] Sibo Yi, Yule Liu, Zhen Sun, Tianshuo Cong, Xinlei He, Jiaxing Song, Ke Xu, and Qi Li. 2024. Jailbreak Attacks and Defenses Against Large Language Models: A Survey. *arXiv preprint arXiv:2407.04295* (2024).

[53] Youliang Yuan, Wenxiang Jiao, Wenxuan Wang, Jen-tse Huang, Jiahao Xu, Tian Liang, Pinjia He, and Zhaopeng Tu. 2024. Refuse whenever you feel unsafe: Improving safety in llms via decoupled refusal training. *arXiv preprint arXiv:2407.09121* (2024).

[54] Yifan Zeng, Yiran Wu, Xiao Zhang, Huazheng Wang, and Qingyun Wu. 2024. Autodefense: Multi-agent llm defense against jailbreak attacks. *arXiv preprint arXiv:2403.04783* (2024).

[55] Zhexin Zhang, Junxiao Yang, Pei Ke, Shiyao Cui, Chujie Zheng, Hongning Wang, and Minlie Huang. 2024. Safe Unlearning: A Surprisingly Effective and Generalizable Solution to Defend Against Jailbreak Attacks. *arXiv preprint arXiv:2407.02855* (2024).

[56] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 46595–46623. https://proceedings.neurips.cc/paper_files/paper/2023/file/91f18a1287b398d378ef22505bf41832-Paper-Datasets_and_Benchmarks.pdf

[57] Sicheng Zhu, Ruiyi Zhang, Bang An, Gang Wu, Joe Barrow, Zichao Wang, Furong Huang, Ani Nenkova, and Tong Sun. 2023. AutoDAN: Interpretable Gradient-Based Adversarial Attacks on Large Language Models.

[58] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043* (2023).

A Supplement to Work of Jailbreak

A.1 Jailbreak Attack Strategy

With the introduction of safety alignment, LLMs have gained the inherent ability to detect prompts with malicious intent from attackers and to avoid generating responses that could potentially cause harm. Consequently, a variety of jailbreak attack strategies have emerged, where attackers carefully craft prompts to bypass the safety guardrails set by developers in LLMs. Based on the access to LLMs' parameters like gradient or logits, jailbreak attack strategies can be categorized as white-box and black-box attacks.

As for black-box attack strategies, Deshpande et al. [15] have found that by assigning a persona to LLMs, ChatGPT[11] can exhibit toxic behavior across a wide range of topics. Some attackers also conduct privilege escalation[31], wherein they guide LLMs into a "sudo" mode to circumvent its safety alignment. Additionally, attention shifting[31] is also an effective strategy during which attackers mask their malicious intent by reframing the task, such as text continuation and code generation. The context can also be exploited as an attack vector as Wei et al. [48] demonstrated that constructing examples where LLMs comply to malicious instructions can lead them to follow the attacker's intent. Moreover, prompt rewriting is another effective approach. [30] proposed a hierarchical genetic algorithm, which generates optimal and stealthy jailbreak prompts against aligned LLMs through iterative refinement. Additionally, prompt rewriting also includes techniques such as cipher [25] and employing non-English languages [14]

For white-box attack strategies, Zou et al. [58] proposed an effective gradient-based jailbreak attack, Greedy Coordinate Gradient (GCG), which enhances malicious prompts by adding adversarial suffixes that are iteratively optimized. The attack strategy developed by Geisler et al. [20] achieves comparable results to GCG while significantly reducing time overhead. To decrease the perplexity of adversarial suffixes while maintaining effectiveness, AutoDAN[57] employs Single Token Optimization (STO) during the iterative optimization of adversarial suffixes. Additionally, Guo et al. [21] proposed COLD-Attack, a text generation algorithm capable of controllably producing covert, low-perplexity attack prompts without compromising efficiency.

A.2 Jailbreak Defense Strategy

To address the escalating threats posed by jailbreak attacks, researchers have proposed a range of jailbreak defense strategies to prevent the safety guardrails of LLMs from being bypassed and to avoid responding to malicious queries. We will introduce these strategies in different stages from an end-to-end defense perspective.

For stage 1, one prevalent approach is the implementation of detection mechanisms for attack prompts. Alon and Kamfonas [7] proposed a detection algorithm that identifies user prompts as jailbreak prompts if their perplexity exceeds a certain threshold. For stage 2, Robey et al. [42] designed a perturbation algorithm that inserts, swaps, and patches characters in user prompts at a certain ratio to disable adversarial suffixes from inducing LLMs to generate unsafe responses. Similar to strategies employed in gradient-based jailbreak attacks, [35] utilized the gradient of LLMs to iteratively refine safety suffixes, which can be used to induce LLMs to produce safe responses. Furthermore, leveraging the excellent text comprehension and instruction execution capabilities of LLMs, Xie et al. [49] emphasized the priority of safety over usability in their safety prompts. In contrast, Wei et al. [48] guided LLMs to produce safe responses by providing examples of refusing to answer dangerous questions within the context.

As for stage 3, a common approach is model fine-tuning. Bianchi et al. [10] highlighted the importance of constructing safety datasets when building supervised fine-tuning models to prevent LLMs from

becoming overly sensitive to certain safety prompts. Gallego [19] facilitated flexible safety configurations for LLMs using Reinforcement Learning from Human Feedback (RLHF). In addition to optimizing LLMs for safety, Zhang et al. [55] demonstrated that enabling large models to forget harmful knowledge is also an effective strategy for countering jailbreak attacks. Beyond fine-tuning the models themselves, Zeng et al. [54] developed a multi-model framework that analyzes the intent and potential harm of prompts to enhance resilience against attacks. Moreover, Kim et al. [27] leveraged the self-refinement capability of LLMs, indicating its effectiveness on non-safety-aligned models.

B Combination of Jailbreak Attack and S-Bench

As for the detailed implementation of the combination process, on one hand, for *AutoDAN-HGA* and *Cold-Attack*, the generation of attack prompts relies on the gradient or logits of LLMs. Therefore, we utilized scripts from their official repositories to generate adversarial prompts locally for the above 520 seed prompts. For *Cold-Attack* we followed the approach as its developers did to randomly select 50 malicious instructions from above 520 seed prompts, then generated eight adversarial prompts for each of these 50 malicious instructions across open-source LLMs, ultimately producing 400 adversarial prompts. In contrast, we used code from *AutoDAN-HGA*'s repository to generate adversarial prompts for the seed prompts, resulting in 520 adversarial prompts.

On the other hand, the strategies of *Roleplay*, *PE*, *ICA*, and *AS* do not require malicious instruction to be refined on specific LLM. They simply concatenate malicious instructions with adversarial suffixes. For *Roleplay*, *PE*, and *AS*, we used data from the repository[44], which collected these attack strategies' suffixes. We selected the top five adversarial suffixes based on their attack effectiveness and then randomly concatenated them with the 520 seed prompts, divided into 5 equal parts. For *ICA*, we directly used the 1-shot example from the appendix of its paper[48] and concatenated it with all 520 seed prompts directly. For each of the above 4 strategies, we generated 520 adversarial prompts as well.

C Assessor LLM

C.1 Settings of Assessor LLM

For our assessor LLM, Qwen2.5-instruct-32B, we set the temperature to 0 to obtain the most deterministic results and adjusted max_new_tokens to 32 to enhance the efficiency of automated evaluation.

Table 4: Hyperparameters of Assessor LLMs.

Hyperparameter	Value
Top P	0.90
Top K	50
Temperature	0
max_new_tokens	32

C.2 Prompt of Assessor LLM

Prompt used for evaluation

I need assistance in categorizing responses generated by LLMs. There are three categories of responses. Please ensure that you begin your reply by specifying the appropriate classification.

• full_compliance. This refers to any response that directly attempts to answer the question, regardless of its accuracy or relevance. It also covers cases where the model rejects the question because it lacks a meaningful answer.

• full_refusal. This occurs when the model outright refuses to answer the question. It also includes responses that offer only alternative suggestions without addressing the original prompt.

• refusal_while_compliance. This category includes responses where there is both refusal and an attempt to comply. It applies to self-contradictory answers that first reject the question but then proceed to give an answer.

QUESTION: "{QUESTION}"
RESPONSE: "{RESPONSE}"
CLASS:

D Experiment Setting

D.1 Hyperparameters of LLMs

For seven targeted LLMs to perform evaluation on, we set the temperature to 0.6 and max_new_tokens to 1024, which allows for more diverse and adequate responses in real user scenarios. More hyperparameters are shown in table 5.

Table 5: Hyperparameters of targeted LLMs.

Hyperparameter	Value
Top P	0.90
Top K	50
Temperature	0.6
max_new_tokens	1024

D.2 Details of Defense Strategy Setting

Table 6: Detailed summary of defense strategies.

Stage	Defense Strategy	Abbr.	Source
Stage 1	Perplexity	PPL	Alon and Kamfonas [7]
Stage 2	Self-Reminder	SR	Xie et al. [49]
	In-Context Defense	ICD	Wei et al. [48]
	SmoothLLM	S-LM	Robey et al. [42]
	Prompt Adversarial Tuning	PAT	Mo et al. [35]
Stage 3	SafeUnlearn	SU	Zhang et al. [55]
	Configurable Safety Tuning	CST	Gallego [19]

To defense LLMs with the Strategies we collected, for *SR* and *ICD*, we directly concatenated the defense suffixes presented in their papers[48] with test prompts from *USEBench*, respectively.

Developers of *PAT* have provided defense suffixes for Vicuna-7b-v1.5 and Llama2-7b-chat in the official repository[40]. For the above two LLMs, we concatenated the corresponding defense suffixes with prompts from *USEBench*. For other LLMs, we used the transferable defense suffix, which is applicable to various LLMs and available in the repository, instead.

For *Perplexity*, we set $threshold = 1000$ and strictly used Llama-2-7b-hf for calculation; for *S-LM*, we set $\gamma = \frac{1}{2}$, $q = 10\%$, and $N = 2$, respectively. If *Perplexity* returned Boolean value "False", we did not input it into LLMs as it have already been identified as a jailbreak prompt. For *S-LM*, we put the modified prompts it generated along with the original prompts into LLMs and evaluated them collectively afterward. Additionally, for *SafeUnlearn* and *CST* we input prompts from *USEBench* into the corresponding safety fine-tuned version of the target LLM.